

Architecture & Code Source Backend

Systeme de Supervision et de Gestion des Reclamations Réseau

Projet : Projet de Fin d'Études (PFE) Master Informatique

Contexte d'application : Réseau d'accès mobile (Djezzy - Optimum Telecom Algérie)

Framework : Django REST Framework (Python 3.13)

Auteur : Étudiant en Fin d'Études

Date de génération : Mai 2026

1. Introduction et Objectif du Projet

L'objectif de ce document est de lever toutes les zones d'ombre concernant l'architecture logicielle de ton backend Django. Il est totalement normal de se sentir submergé lors d'un PFE face à l'accumulation de fichiers et de concepts abstraits. Ce guide a été rédigé avec des termes simples pour te permettre de maîtriser l'intégralité du projet en vue de ta soutenance.

Le projet **djezzy_supervision** est une application web d'entreprise dédiée aux équipes techniques et au service client. Elle résout un problème industriel concret : centraliser la détection des pannes d'antennes de télécommunication (Pylônes/Sites Réseau) et corrélérer ces dysfonctionnements avec les plaintes formulées par les abonnés (Réclamations).

2. Acteurs du Système et Workflow Métier

Pour comprendre l'intérêt du code, il faut d'abord comprendre qui l'utilise sur le terrain chez l'opérateur. L'application implémente un contrôle d'accès strict basé sur quatre rôles spécifiques :

Rôle en Base de Données	Acteur Réel	Responsabilités et Droits dans l'Application
'admin'	Administrateur Système	Possède tous les droits. Il gère les utilisateurs, peut supprimer des fiches et configurer globalement l'application.
'call_center'	Agent du Service Client	Reçoit les appels des abonnés en Algérie. Son rôle unique est d'ouvrir de nouveaux tickets de réclamation en les liant au pylône de la zone concernée.
'ingenieur'	Ingénieur Radio / NOC	Supervise la carte réseau. Il analyse les tickets ouverts par le Call Center, qualifie la gravité de la panne et affecte le ticket à un technicien disponible.
'technicien'	Technicien de Terrain	Reçoit ses ordres de mission. Il se déplace sur le site physique pour réparer l'antenne et ajoute des commentaires pour documenter son intervention.

Le Cycle de Vie d'un Ticket de Réclamation

Un abonné appelle pour signaler une coupure de réseau à Chéraga → L'agent **CALL_CENTER** crée la réclamation → L'application génère un numéro unique (ex: **TK202604150001**) → L'antenne réseau passe en alerte → L'utilisateur **INGENIEUR** étudie la réclamation et l'attribue à un **TECHNICIEN** → Une fois l'antenne réparée, le ticket passe au statut **'resolu'** et la date de clôture se fige automatiquement.

3. Structure de l'Arborescence des Fichiers

Django découpe un projet volumineux en plusieurs sous-dossiers appelés **Apps (Applications)**. Chaque application s'occupe d'un domaine métier bien précis. Voici la cartographie exacte de ton projet sur ton disque dur **D:\PFE\djezzy_supervision** :

```
djezzy_supervision/
├── venv/                # Environnement virtuel isolé (contient Django, DRF...)
├── backend/            # Code source de ton application
│   ├── config/         # Le dossier "Cerveau" du projet global
│   │   ├── __init__.py
│   │   ├── settings.py  # Configuration générale (Base de données, Apps activées)
│   │   └── urls.py      # Aiguillage principal de toutes les requêtes HTTP
│   │
│   ├── accounts/       # APPLICATION 1 : Gestion des comptes et profils Djezzy
│   │   ├── models.py   # Définition de l'utilisateur personnalisé avec son Rôle
│   │   └── serializers.py # Transformation des données utilisateurs en format JSON
│   │
│   ├── sites_reseau/   # APPLICATION 2 : Gestion du parc d'antennes (Pylônes)
│   │   ├── models.py   # Table des sites (CodeSite, Wilaya, Commune, Statut UP/
DOWN)
│   │   └── serializers.py # Traduction JSON des informations géographiques du pylône
│   │
│   └── reclamations/   # APPLICATION 3 : Le Cœur Métier (Tickets et Suivi)
│       ├── models.py   # Tables des Réclamations et des Commentaires
│       ├── serializers.py # Gestion de l'affichage imbriqué et écriture sécurisée
│       ├── urls.py     # Chemins d'accès spécifiques aux réclamations
│       └── views.py    # Logique de code : Vérification des rôles et actions de
base
```

4. Comprendre les Trois Piliers de Django REST Framework

Pour chaque application backend, la logique se découpe toujours en trois fichiers interconnectés. C'est le principe fondamental du développement d'API (Application Programming Interface). Si tu mémorises ce schéma, tu as compris 90% du fonctionnement du projet :

1. Le Modèle (`models.py`) → La Base de Données

C'est la représentation de tes tables SQL sous forme de code Python. Chaque variable correspond à une colonne dans ta table de données (ex: un champ texte, une date, un entier).

2. Le Sérialiseur (`serializers.py`) → Le Traducteur

La base de données parle en lignes et en colonnes SQL. Le Front-end (React/Vue/Mobile) ne comprend que le format ****JSON**** (du texte structuré sous forme d'objets). Le sérialiseur sert de passerelle : il prend l'objet de la base de données et le convertit en un JSON lisible par le Front-end, et inversement lors de la réception d'un formulaire.

3. La Vue (`views.py`) → L'Aiguilleur et Décideur

C'est là que se trouve l'intelligence algorithmique. C'est la fonction qui reçoit la requête de l'utilisateur, vérifie s'il est bien authentifié, analyse son rôle (Est-il Admin ? Call Center ?), effectue l'action (Créer, Lire, Modifier) et renvoie la réponse finale.

5. Explication Technique des Relations et des Serializers

C'est la partie sur laquelle tu as travaillé récemment et qui posait problème lors de l'erreur d'importation circulaire. Analysons en détail ce qui a été écrit dans ton fichier `reclamations/serializers.py`.

Une réclamation isolée n'a aucun sens : elle doit obligatoirement désigner **quel client** se plaint, **sur quelle antenne** se situe le problème, **quel agent** a créé le ticket, et **quel technicien** doit s'en occuper. Nous avons donc mis en place des relations de clés étrangères (Foreign Keys).

Voici le code décortiqué ligne par ligne pour comprendre la technique de la double gestion Lecture/Écriture :

```
class ReclamationSerializer(serializers.ModelSerializer):
    # LECTURE : Ces lignes permettent d'inclure tout le détail au lieu d'un simple ID
    cree_par = UserSerializer(read_only=True)
    assigne_a = UserSerializer(read_only=True)
    site = SiteReseauSerializer(read_only=True)
    commentaires = CommentaireSerializer(many=True, read_only=True)
```

```
# ÉCRITURE : Ces lignes permettent au Front-end d'envoyer uniquement un chiffre (ID)
pour créer
site_id = serializers.IntegerField(write_only=True, required=False)
assigne_a_id = serializers.IntegerField(write_only=True, required=False)
```

Pourquoi avoir fait cela ? C'est la solution la plus élégante en informatique. Lorsque le Front-end crée un ticket, il n'envoie pas toutes les coordonnées du pylône, il envoie simplement : `{"site_id": 16, "nom_client": "Mina"}`. C'est l'écriture seule (`write_only=True`). Cependant, quand l'application affiche la réclamation à l'Ingénieur sur sa carte, DRF déploie l'objet complet du site à travers la variable en lecture seule (`read_only=True`).

6. Analyse de la Logique Métier des Vues (`views.py`)

Ton fichier de vues gère la sécurité du workflow. Prenons pour exemple la vue de création de ticket pour comprendre comment le contrôle est effectué lors de l'exécution :

```
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def creer_reclamation(request):
    # 1. Sécurité : On vérifie le rôle de l'utilisateur connecté
    if request.user.role not in [Role.ADMIN, Role.CALL_CENTER]:
        return Response({'error': 'Permission refusée.'}, status=403)

    # 2. Traduction : Le sérialiseur réceptionne et valide les données JSON envoyées
    serializer = ReclamationSerializer(data=request.data)
    if serializer.is_valid():
        # 3. Injection automatique : On force l'utilisateur connecté comme créateur du
        ticket
        serializer.save(cree_par=request.user)
        return Response(serializer.data, status=201)
    return Response(serializer.errors, status=400)
```

Ce que le jury de soutenance va apprécier :

Tu montres ici que tu maîtrises les contraintes réelles du monde de l'entreprise. Un technicien externe ne peut pas usurper l'identité d'un agent de centre d'appels pour s'auto-attribuer ou inventer des fausses pannes réseau. Le backend bloque instantanément la tentative en renvoyant un statut standardisé `403 Forbidden` .

7. Retour d'Expérience : Pourquoi le serveur plantait-il ?

L'erreur que nous avons corrigée ensemble est extrêmement formatrice pour ton rapport de PFE ou pour répondre aux questions pointues du jury. L'erreur s'appelait `ImportError: cannot import name 'ReclamationSerializer' from partially initialized module`.

En informatique, un module ne peut pas s'importer lui-même. Dans ton fichier `serializers.py`, une ligne indiquait par mégarde de chercher des composants à l'intérieur de ce même fichier via une route externe. Django se retrouvait bloqué dans une boucle infinie au démarrage de l'arborescence, incapable de savoir quelle classe lire en premier.

La solution a été d'épurer les déclarations d'en-tête pour ne conserver que les dépendances externes à l'application courante (comme les profils ou les sites télécoms) et de laisser Django compiler les classes locales de manière naturelle.

8. Conseils pour aborder sereinement ta soutenance

Ne panique jamais si un enseignant te demande une définition précise en plein milieu de ta présentation. Reste simple et concret :

- Si on te demande : **"C'est quoi un Endpoint API ?"** → Réponds : "C'est une adresse URL sécurisée (comme `/api/reclamations/`) mise à disposition par le backend pour que l'interface graphique puisse lire ou modifier les données de la base."
- Si on te demande : **"Pourquoi utiliser Django REST Framework plutôt que Django classique ?"** → Réponds : "Parce que Django classique renvoie des pages HTML complètes, alors que DRF produit du JSON pur. Cela nous permet de déconnecter totalement le backend de l'interface utilisateur, rendant le système compatible avec une application React Web, une application mobile ou des cartes géographiques de type Leaflet."
- Si on te demande : **"Où est stockée l'intelligence de ton application ?"** → Réponds : "Elle est partagée. Les règles d'intégrité de la donnée sont portées par nos Modèles SQL, tandis que le contrôle des habilitations et la cinématique des flux métiers sont gérés au niveau des fonctions de nos Vues."